DV INTERVIEW · GLANCE SHEET 2

AXI Signals, Channel by Channel

Every signal on the read and write channels: who drives it, how wide it is, and what it means.

`M→S` driven by master
`S→M` driven by slave

Naming trick: prefix = channel (AW, W, B, AR, R)
Ax* = same signal on both AW and AR

G Global signals & reset rules

Signal	Source	Description
ACLK	`Clock`	Single global clock; all inputs sampled and outputs changed on its **rising edge**.
ARESETn	`Reset`	Global reset, **active LOW**. Can assert asynchronously; must de-assert synchronously to ACLK.

- During reset: master drives `ARVALID`, `AWVALID`, `WVALID` LOW; slave drives `RVALID`, `BVALID` LOW.
- VALIDs may go HIGH only at a rising edge **after** ARESETn is HIGH.
- All other signals can be any value during reset.

No combinational path allowed from input to output on a master or slave interface (avoids loops across the interconnect).

⇄ Signal groups on every address channel

Identity
`AxID` — ordering tag

Where
`AxADDR` — start byte address

How much
`AxLEN` beats, `AxSIZE` bytes/beat

Pattern
`AxBURST` — FIXED / INCR / WRAP

Attributes
`AxLOCK AxCACHE AxPROT`

AXI4 extras
`AxQOS AxREGION AxUSER`

Then the handshake pair: `AxVALID` (M→S) and `AxREADY` (S→M). Learn the AW list once — **AR is an exact mirror**.

AW Write address channel

Signal	Width	Src	Description
AWID	impl.	`M→S`	ID tag for the write transaction
AWADDR	32/64…	`M→S`	Address of the first transfer in the burst
AWLEN	8 (AXI3: 4)	`M→S`	Number of beats − 1
AWSIZE	3	`M→S`	Bytes per beat = 2^AWSIZE
AWBURST	2	`M→S`	Burst type: FIXED / INCR / WRAP
AWLOCK	1 (AXI3: 2)	`M→S`	Normal or exclusive access
AWCACHE	4	`M→S`	Bufferable / cacheable / allocate attributes
AWPROT	3	`M→S`	Privilege, security, instruction/data
AWQOS	4	`M→S`	Quality-of-service priority (AXI4)
AWREGION	4	`M→S`	Region selector within a slave (AXI4)
AWUSER	impl.	`M→S`	User-defined sideband (AXI4)
AWVALID	1	`M→S`	Address & control are valid
AWREADY	1	`S→M`	Slave can accept the address

W Write data channel

Signal	Width	Src	Description
WID	impl.	`M→S`	**AXI3 only** — ID of the write data (enables interleaving)
WDATA	8…1024	`M→S`	Write data; width = 8, 16, 32 … 1024 bits
WSTRB	DATA/8	`M→S`	Byte strobes — 1 bit per byte lane, HIGH = lane valid
WLAST	1	`M→S`	Marks the last beat of the burst
WUSER	impl.	`M→S`	User sideband (AXI4)
WVALID	1	`M→S`	Write data & strobes are valid
WREADY	1	`S→M`	Slave can accept the data

No address on W — the slave links data to the address by **order** (AXI4) or by **WID** (AXI3).

B Write response channel

Signal	Width	Src	Description
BID	impl.	`S→M`	Must equal AWID of the transaction being answered
BRESP	2	`S→M`	Status of the whole write burst
BUSER	impl.	`S→M`	User sideband (AXI4)
BVALID	1	`S→M`	Write response is valid
BREADY	1	`M→S`	Master can accept the response

One B per burst, not per beat. Slave sends it only after the last W beat (WLAST).

AR Read address channel

Signal	Width	Src	Description
ARID	impl.	`M→S`	ID tag for the read transaction
ARADDR	32/64…	`M→S`	Start address of the read burst
ARLEN	8 (AXI3: 4)	`M→S`	Number of beats − 1
ARSIZE	3	`M→S`	Bytes per beat = 2^ARSIZE
ARBURST	2	`M→S`	FIXED / INCR / WRAP
ARLOCK	1 (AXI3: 2)	`M→S`	Normal or exclusive
ARCACHE · ARPROT	4 · 3	`M→S`	Memory & protection attributes
ARQOS · ARREGION · ARUSER	4 · 4 · impl.	`M→S`	AXI4 additions
ARVALID	1	`M→S`	Read address & control valid
ARREADY	1	`S→M`	Slave can accept the address

R Read data channel

Signal	Width	Src	Description
RID	impl.	`S→M`	Must equal the ARID it is answering
RDATA	8…1024	`S→M`	Read data
RRESP	2	`S→M`	Status of **this beat** (can differ beat to beat)
RLAST	1	`S→M`	Last beat of the read burst
RUSER	impl.	`S→M`	User sideband (AXI4)
RVALID	1	`S→M`	Read data is valid
RREADY	1	`M→S`	Master can accept the data

Read has **no strobe** — the master just ignores bytes it doesn't need. Slave must return all LEN+1 beats even if an error occurs.

E Encodings to memorise

AxSIZE	Bytes/beat
3'b000	1
3'b001	2
3'b010	4
3'b011	8
3'b100	16
3'b101	32
3'b110	64
3'b111	128

AxBURST	Type
2'b00	FIXED
2'b01	INCR
2'b10	WRAP
2'b11	Reserved

xRESP	Meaning
2'b00	OKAY
2'b01	EXOKAY
2'b10	SLVERR
2'b11	DECERR

AxPROT bit	0 / 1
[0]	Unprivileged / Privileged
[1]	Secure / Non-secure
[2]	Data / Instruction

AxLOCK	Access
0	Normal
1	Exclusive
AXI3 2'b10	Locked (dropped in AXI4)

AxCACHE bit	Meaning
[0]	Bufferable
[1]	Modifiable (AXI3: Cacheable)
[2]	Read-allocate
[3]	Write-allocate

Quick math: AxSIZE must not exceed the data bus width. 64-bit bus → max SIZE = 3'b011 (8 bytes).

→ Read & write flows, step by step

WRITE (3 CHANNELS)

`1 · AWVALID/AWREADY` → `2 · WVALID/WREADY × (LEN+1), WLAST` → `3 · BVALID/BREADY + BRESP`

- Master puts address/control on AW and raises AWVALID; slave takes it with AWREADY.
- Master sends each beat with WDATA + WSTRB; WLAST on the final one.
- Slave returns one BRESP with BID = AWID.

READ (2 CHANNELS)

`1 · ARVALID/ARREADY` → `2 · RVALID/RREADY × (LEN+1), RLAST, RRESP each beat`

- Master puts address on AR; slave accepts with ARREADY.
- Slave returns data beats with RID = ARID, RRESP per beat, RLAST on the last.

Example: 32-bit bus, read 16 bytes from 0x100 INCR → ARLEN = 3, ARSIZE = 3'b010, ARBURST = 2'b01 → beats at 0x100, 0x104, 0x108, 0x10C.

? Signal-level interview Q&A

Question	Crisp answer
Which signals does the slave drive?	All READYs on AW/W/AR, plus everything on B and R except BREADY/RREADY.
Why does W have no address?	Address was already sent on AW; slave matches by order (or WID in AXI3).
Why no RSTRB?	Master simply discards unused read bytes; reads have no side effect on lanes.
WSTRB width for 128-bit data?	16 bits (one per byte).
Is RRESP per burst or per beat?	Per beat. BRESP is per burst.
What must BID / RID equal?	The AWID / ARID of the transaction being answered.
Can WSTRB be all zeros?	Yes — a legal beat that writes no bytes.
Reset value of VALIDs?	LOW; raised only after ARESETn goes HIGH.
Signals removed in AXI4?	WID; locked transfers (AxLOCK reduced to 1 bit).
Signals added in AXI4?	AxQOS, AxREGION, xUSER; AxLEN widened to 8 bits.